

Comp 579: Final Project

Dreamer V1 Reproduction

Nathan Claret

October 6, 2026

Abstract

This report presents the final project I completed as part of the COMP 579 course. In this project, I attempted and successfully reimplemented the Dreamer model in its first version, Dreamer V1, originally introduced in Hafner et al. [2020]. Dreamer is a model-based reinforcement learning agent that learns a world model from experience collected in the environment. This world model encodes observations into a compact latent space, learns to predict future transitions and rewards, and then allows the agent to learn its behavior by imagining trajectories directly within this latent space. The actor and the critic are thus trained from these imagined trajectories, using analytic gradients propagated through the learned latent dynamics. My goal was to reproduce the main components of Dreamer V1 — namely the latent model, the reward model, the actor, and the critic — in order to better understand how latent imagination can be leveraged to learn effective behaviors in control environments.

1 Introduction

Reinforcement learning has made remarkable progress in recent years, largely driven by model-free algorithms such as PPO and SAC. However, these approaches suffer from limited sample efficiency, often requiring millions of interactions with the environment to reach satisfactory performance, a prohibitive constraint in real-world settings where data collection is costly or risky.

Model-based algorithms, and in particular *world models*, offer a promising alternative. Their underlying intuition draws directly from human cognition: we maintain an internal representation of our environment that lets us imagine the consequences of our actions without actually executing them. This ability to plan in a mental space drastically reduces the number of experiences needed to master a task. It is precisely this elegance — algorithmically reproducing a mechanism observable in nature — that drew me to this family of methods.

The Dreamer algorithm, introduced by Hafner et al. [2020], embodies this idea: it learns a latent model of the environment dynamics and optimizes a policy entirely within this imagined space, without any new interaction with the

simulator during policy learning. Successive iterations of this family — up to Dreamer v4 Hafner et al. [2025] — have surpassed the state of the art on several demanding benchmarks while preserving excellent sample efficiency.

This project reproduces Dreamer v1 in order to gain a deeper understanding of the mechanisms behind this approach. By implementing the key components myself — representation model, recurrent latent dynamics, reward predictor, and policy learning in imagination — I aim to experimentally validate the results reported in the original paper and identify the practical challenges of this type of architecture.

2 Background

POMDP with visual observations. The environment is modeled as a Partially Observable Markov Decision Process (POMDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, p, r)$, where the agent does not receive the state s_t but only an observation o_t — here an RGB image. The temporal dependence of observations makes it necessary to learn an internal *state* summarizing the history $(o_{1:t}, a_{1:t-1})$, sufficient to predict the dynamics and the reward.

World model. A world model is a learned generative model that approximates the environment dynamics in a compact latent space. It allows *generating* synthetic (imagined) trajectories without interacting with the real simulator, which drastically improves sample efficiency. Policy learning can then be carried out entirely within this latent space.

Recurrent State-Space Model (RSSM). Dreamer builds on the RSSM, which decomposes the latent state into two complementary components: a deterministic state h_t produced by a GRU, and a stochastic state z_t drawn from a Gaussian distribution. The model defines three key functions:

- **Recurrent model:** $h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1})$
- **Prior (transition):** $\hat{z}_t \sim p_\phi(\hat{z}_t | h_t)$
- **Posterior (encoding):** $z_t \sim q_\phi(z_t | h_t, o_t)$

To these are added an observation decoder $p_\phi(o_t | h_t, z_t)$ and a reward predictor $p_\phi(r_t | h_t, z_t)$. The whole model is trained jointly by maximizing an ELBO combining a reconstruction term, a reward prediction term, and a KL divergence between prior and posterior acting as a regularizer. Gaussian reparametrization allows gradients to be propagated through the stochastic states.

Latent imagination learning. Once the world model has been learned, Dreamer trains an actor $\pi_\psi(a_t | h_t, z_t)$ and a critic $V_\xi(h_t, z_t)$ on trajectories imagined entirely in the latent space, starting from initial states drawn from the

replay buffer. The returns used are λ -returns Sutton and Barto [2020], which interpolate between estimates at different horizons:

$$V_\lambda(h_t, z_t) = (1 - \lambda) \sum_{n=1}^{H-1} \lambda^{n-1} V_N^n + \lambda^{H-1} V_N^H, \quad (1)$$

where V_N^n is the n -step return. The actor is optimized by gradient descent through the differentiable dynamics of the world model — a strategy made possible by reparametrization — which distinguishes Dreamer from previous model-based approaches based on planning or model-free policy gradients.

3 Methodology

World model components. The implementation is organized around seven jointly trained modules (Figure 1). First, an **Encoder** (4-layer CNN with ELU activations) projects an observation o_t into a dense vector retaining its essential information. A **Recurrent model** (GRUCell) maintains the deterministic state h_t summarizing the history, via $h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1})$. A **Prior** (MLP) predicts the latent transition $\hat{z}_t \sim p_\phi(\cdot | h_t)$ *without* observing o_t — this is the module that enables imagination. A **Posterior** (MLP) infers $z_t \sim q_\phi(\cdot | h_t, e_t)$ from the encoded observation e_t , providing accurate targets for the prior through the KL divergence. A **Decoder** (transposed CNN, symmetric to the encoder) reconstructs o_t from the full state $s_t = (h_t, z_t)$, forcing the latter to be informative. A **Reward model** (MLP) predicts r_t from s_t , anchoring the latent representation in the semantics of the task. Finally, a **Continue model** (MLP, disabled here) predicts the probability of non-termination.

The stochastic state z_t follows a diagonal Gaussian sampled via reparametrization to enable backpropagation through the latent dynamics.

World model loop. Given a sequence $(o_{0:T}, a_{0:T-1}, r_{0:T-1})$ drawn from the replay buffer, the RSSM is unrolled recursively (Figure 1): the recurrent model updates h_t , the posterior infers z_t from h_t and $e_t = \text{Encoder}(o_t)$, and the full state $s_t = (h_t, z_t)$ feeds the decoder, the reward model and the continue model. The total loss combines reconstruction, reward prediction, and KL divergence with free nats τ :

$$\mathcal{L}_{wm} = \mathcal{L}_{recon} + \mathcal{L}_{reward} + \beta \cdot \max(\text{KL}(q_\phi \| p_\phi), \tau). \quad (2)$$

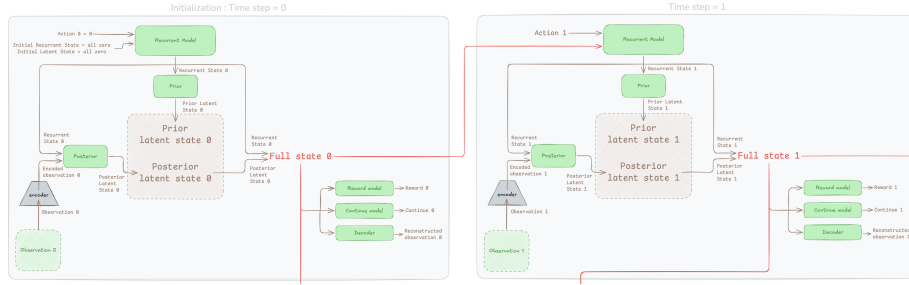


Figure 1: Diagram of world model training. At each step of the sequence, the recurrent model updates h_t , the posterior infers z_t from the encoded observation, and the full state (h_t, z_t) feeds the reward model, the continue model, and the decoder. The prior, which does not see o_t , is regularized to match the posterior through the KL divergence. This loop is repeated until *BatchLength* is reached.

Behavior components. Two additional modules learn the policy in imagination. An **Actor** (MLP) outputs a Gaussian distribution over actions, constrained by *tanh-squashing* to lie within $[-1, 1]$. Sampling is performed via reparametrization to enable analytic gradients. A **Critic** (MLP) estimates the state value $V_\xi(s_t)$, trained by regression on the λ -returns computed along the imagined trajectory.

Imagination loop. Starting from initial states s_0 drawn from the previous phase, a trajectory is generated entirely in the latent space (Figure 2): at each step, the actor produces a_t , the recurrent model updates h_{t+1} , and the *prior* samples z_{t+1} — without any real observation. The reward model and the critic evaluate the trajectory. The actor maximizes the λ -returns through gradient descent across the differentiable dynamics, while the critic regresses on the same targets, *detached* from the graph.

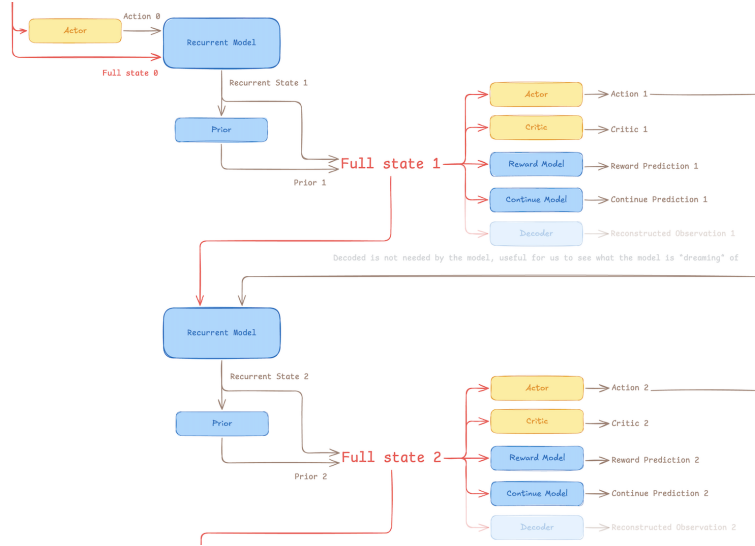


Figure 2: Diagram of behavior training. Starting from an initial full state produced by the world model, an imagined trajectory is generated: the actor outputs a_t , while the recurrent model and the *prior* produce (h_{t+1}, z_{t+1}) without any real observation. The decoder, shown in transparency, is not used during policy training but helps visualize what the model is “imagining”. This loop is repeated until *ImaginationHorizon* is reached.

Global training loop. After a bootstrap phase using a random policy, training alternates between several gradient steps and one episode of interaction with the environment (`replay_ratio`). Each batch contains sequences sampled from the replay buffer. Evaluation is performed periodically on a deterministic episode (mean action rather than sampled) 3.

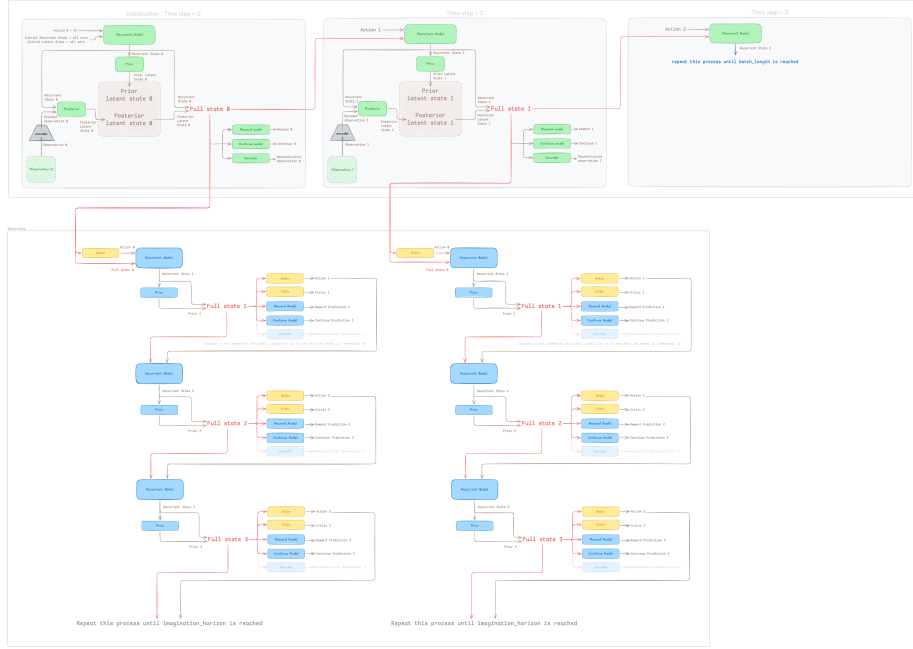


Figure 3: Overview of the full Dreamer v1 pipeline, combining the two phases described above. **Top**: world model training is unrolled over `batch_length` steps, consuming a real sequence drawn from the replay buffer. **Bottom**: behavior training starts from each *full state* produced by this first phase and generates, in parallel for each initial state, an imagined trajectory of length `imagination_horizon` where only the *prior* (and not the posterior) is used to produce successive latent states. The actor, the critic, the reward model, and the continue model are evaluated at each imagined step, while the decoder, shown in transparency, plays no role in policy learning. This representation highlights the central role of the full state as an *interface* between the two phases: it carries all the information needed to move from the real world to the imagined one.

4 Experimental Results

The evaluation is conducted on the **cartpole-swingup** task from the DeepMind Control Suite Tassa et al. [2018], a continuous benchmark in which the agent must swing up and then stabilize a pendulum starting from a downward position. Observations are 64×64 RGB images rendered by the simulator, and the action space is one-dimensional and continuous, $a_t \in [-1, 1]$. All episodes last exactly 1000 steps with no early termination — which justifies disabling the continue predictor, in line with Hafner et al. [2020]. The full set of hyperparameters used is detailed in Appendix A.

Performance. Figure 4 shows the evolution of the average per-episode score during training. After a bootstrap phase in which the score stays around 70 (near-random policy), the agent begins to learn around 20,000 gradient steps and reaches a first plateau near 200 between 40k and 90k steps. A second phase of clear improvement begins after 100,000 steps, raising the average score to approximately 500 by 130,000 gradient steps, which corresponds to roughly 1.3M environment steps given `replay_ratio` = 100.

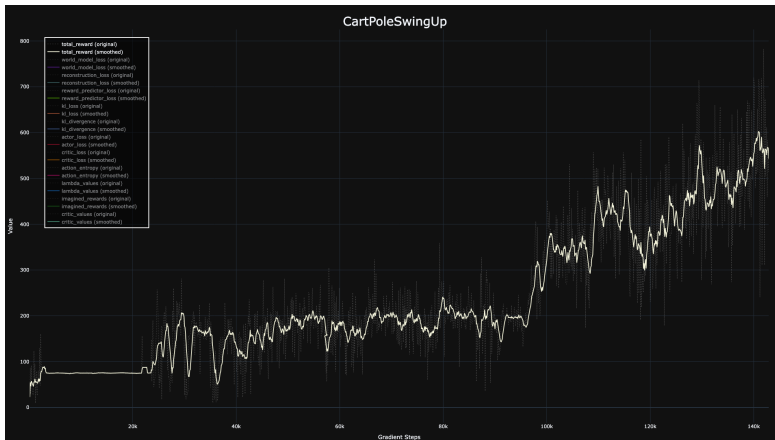


Figure 4: Evolution of the average per-episode score on `cartpole-swingup` during training. The dashed curve shows the raw score, while the solid curve shows its smoothed version. Three phases can be distinguished: bootstrap with a near-random policy (~ 70), a first plateau around 200 between 40k and 90k gradient steps, and a second phase of progression toward ~ 550 after the free nats adjustment at 90k steps.

Comparison with the original paper. Hafner et al. [2020] report a score of approximately 700 after 1M environment steps on the same task. The observed gap is primarily explained by the reduced capacity of our architecture: we use MLPs of size 200 (versus 400) and a recurrent state of dimension 200 (versus 512), a choice dictated by the available compute constraints. Moreover, the learning trajectory suggests that the score would continue to improve with an extended training budget, with no sign of saturation at 140k steps.

Learning dynamics. The two-stage progression, a plateau around 200 followed by a jump toward 500, is characteristic of `cartpole-swingup`. The first plateau corresponds to an agent that keeps the pendulum oscillating without reliably reaching the vertical position, whereas the second phase reflects the discovery of the full swing-up and stabilization strategy. This late transition illustrates the difficulty of credit assignment in this task, where the maximum reward is obtained only within a narrow region of the state space. Qualitatively, the

best model obtained successfully swings the pole up and keeps it nearly vertical for almost the entire episode; a demonstration video is provided in the project submission folder in MP4 format.

World model diagnostics. The reconstruction loss (Appendix B, Figure 6) decreases rapidly during the first 20,000 gradient steps before stabilizing, indicating that the decoder faithfully captures the visual structure of observations from the earliest phases. More importantly, Figure 5 compares the reward actually obtained by the agent with the one predicted by the reward model in its imagined trajectories: the two curves track each other closely throughout training, with only a small residual gap even during phases of rapid progress. This agreement provides empirical evidence that the world model becomes a faithful simulator of the real dynamics — a necessary condition for the policy optimized *in imagination* to transfer effectively to the environment, and one that validates *a posteriori* the philosophy of Dreamer.

A notable observation concerns the role of the free nats threshold τ . With the recommended value $\tau = 3.0$, the KL divergence plateaus around 2.5–2.8 (Appendix B, Figure 7), remaining permanently below the threshold and thus completely neutralizing the regularizer. The posterior and the prior are then free to diverge, which degrades the predictive quality of the prior — yet this prior is essential for imagination. Observing a stagnation of the score, I lowered τ to 1.0 around 90,000 gradient steps; the KL then drops rapidly to ≈ 1.0 and stabilizes there, and the score soon begins its second phase of progression up to ~ 550 . This correlation suggests that an overly permissive free nats threshold is suboptimal for this task Hafner et al. [2022].

5 Conclusion and Future Work

This project successfully reproduces Dreamer v1 on the `cartpole-swingup` task. Despite a deliberately reduced architecture, the agent reaches an average score of around 550, and the close match between imagined and real rewards empirically confirms that the policy optimized in latent space transfers faithfully to the environment — thereby validating the core philosophy of world models.

Several directions remain to be explored. A first improvement concerns the *replay buffer*: the current sampling function (also used in Hafner et al. [2020]) can return sequences spanning two consecutive episodes, introducing inconsistent transitions that hurt world model learning. Strictly filtering these cases should improve stability, especially in early training. A more ambitious direction would be to replace the Gaussian latent state with a *discrete* (categorical) representation, as introduced in Dreamer v2 Hafner et al. [2022]: this seemingly minor change yielded substantial gains in performance and stability, and would constitute a natural transition toward later versions of the algorithm. A systematic hyperparameter search — network sizes, free nats, imagination horizon, replay ratio — would also likely close the gap with the original paper, but proved impractical on personal hardware: a single training run with the

chosen configuration required over 40 hours of compute. Access to more powerful or cloud-based resources would lift this limitation and constitutes a natural extension of this work.

6 Contributions and Acknowledgements

As the sole member of the group, I carried out the entire project on my own. I would nonetheless like to warmly thank the YouTube channel “Inexperienced Me”, whose videos greatly helped me build a clearer understanding of the Dreamer models.

GitHub repo of my code: Dreamer repo

YouTube link to my explanatory video: Dreamer video

References

- Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to Control: Learning Behaviors by Latent Imagination, March 2020.
- Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering Atari with Discrete World Models, February 2022.
- Danijar Hafner, Wilson Yan, and Timothy Lillicrap. Training Agents Inside of Scalable World Models, September 2025.
- Richard S. Sutton and Andrew Barto. *Reinforcement Learning: An Introduction*. Adaptive Computation and Machine Learning. The MIT Press, Cambridge, Massachusetts London, England, second edition edition, 2020. ISBN 978-0-262-03924-6.
- Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, Timothy Lillicrap, and Martin Riedmiller. Deepmind control suite, 2018. URL <https://arxiv.org/abs/1801.00690>.

A Hyperparameters

Table 1: Hyperparameters used to train Dreamer v1 on `cartpole-swingup`.

Parameter	Value
<i>Architecture</i>	
Encoded observation size	512
Recurrent state h_t dimension	200
Stochastic state z_t dimension	30
CNN base depth (encoder/decoder)	16
Conv. layers (encoder/decoder)	4
Kernel / stride / padding (CNN)	3 / 2 / 1
MLP hidden layers	2
MLP hidden size	200
Activation	ELU
Continue model enabled	False
<i>Optimization</i>	
World model learning rate	2×10^{-4}
Actor learning rate	4×10^{-5}
Critic learning rate	1×10^{-4}
Optimizer	Adam
Gradient clipping (norm)	100
Batch size	32
Batch length (sequence)	64
<i>Learning</i>	
Discount γ	0.99
λ -return λ	0.95
Imagination horizon H	12
Free nats τ	1.0
KL scale β	1.0
Entropy scale	0.0003
Replay buffer capacity	50,000
Bootstrap episodes (random policy)	5
Replay ratio (gradient steps / episode)	100
Total gradient steps	145,000
Checkpoint interval (gradient steps)	1,000

B Additional Training Metrics

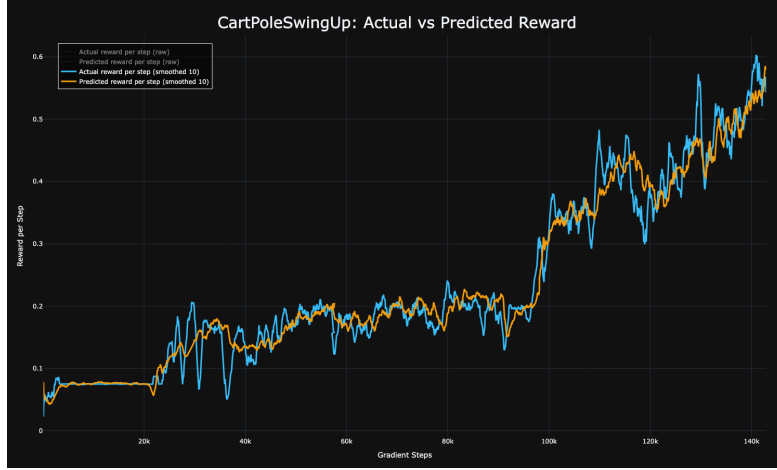


Figure 5: Real reward (blue) vs. reward predicted by the reward model in imagination (orange), averaged per step. The close agreement between the two curves throughout training shows that the world model becomes a faithful simulator — a necessary condition for optimization in imagination to transfer to the real environment.

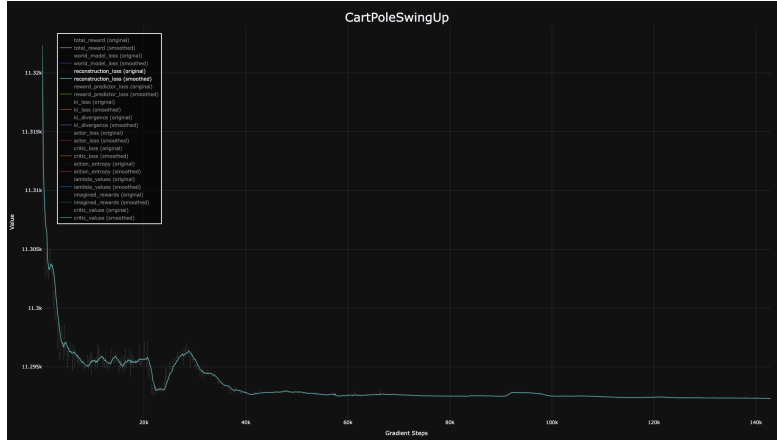


Figure 6: Decoder reconstruction loss during training. The rapid decrease over the first 20,000 gradient steps, followed by a stable plateau, indicates that the world model captures the visual structure of the observations from the earliest phases of training.

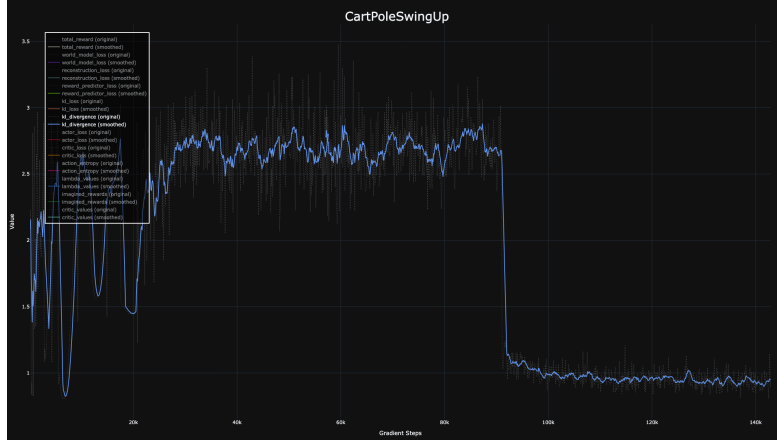


Figure 7: KL divergence between posterior and prior during training. With $\tau = 3.0$ (before 90k steps), the KL plateaus around 2.5–2.8 and the term $\max(\text{KL}, \tau)$ remains saturated at τ , disabling regularization. After lowering τ to 1.0, the KL quickly drops to ≈ 1.0 , restoring an effective gradient signal on the prior.